Nitin

testing project 2

May 29, 2026

Confidentiality and Distribution Restrictions

The conclusions and recommendations in this report represent the opinions of SecurifyAI.

Determinations of appropriate corrective action(s) are the responsibility of the entity receiving the report.

This report and/or any other materials furnished by SecurifyAI in connection with this engagement is confidential and may not be duplicated, modified or otherwise reproduced and distributed without the express prior written consent of SecurifyAI or Nitin.

Because this work may contain copyrighted images or other material, permission from the copyright holder may also be necessary if you wish to reproduce.

Table of Contents

Introduction

As part of an ongoing security program, Nitin identified the need to conduct an application security assessment of its Web application & APIs.

The following lists the objectives of this assessment:

- Determine the overall security posture of the application
- Provide a list of key findings and recommendations for remediation

This report includes the following parameters and results of the assessment:

- SecurifyAI's approach to the assessment
- Assessment scope
- Key findings listed with their qualitative risk assessment
- Detailed recommendations for each finding
- A remediation plan

Approach

Securify performed a Runtime Application Vulnerability Assessment of the Nitin's web application, associated APIs, and all components defined within the assessment scope.

The assessment was conducted using industry-accepted methodologies, primarily based on the OWASP Web Security Testing Guide (WSTG) and OWASP ASVS, and involved both manual testing and automated analysis.

Runtime Application Vulnerability Assessment

The Runtime Application Vulnerability Assessment involved detecting security vulnerabilities through detailed examination and testing of the application in a runtime environment. This assessment emulates an attack by a skilled adversary in a controlled setting and allows Nitin to ascertain the kinds of vulnerabilities that may be realistically exploited. A Runtime Application Vulnerability Assessment includes the following phases:

- **Information Gathering** – The application was reviewed as an anonymous, authenticated, and privileged user to understand differences in access and behavior. Technologies, frameworks, APIs, and third-party integrations were also identified to support targeted testing.
- **Authentication Testing** – Authentication mechanisms were evaluated to determine the strength of login controls, password policies, and account recovery processes. Protections against brute-force attempts and session takeover scenarios were also reviewed to ensure users are securely authenticated.
- **Authorization Testing** – Tests were conducted to identify weaknesses in access control, including attempts to access other users' data or privileged functionality.
- **Session Management** – Session handling was assessed to ensure secure creation, storage, and invalidation of session tokens.
- **Input Validation Attacks** – User-controlled input fields were tested with malformed and malicious data to identify injection flaws and logic bypasses. This included attempts to exploit unexpected behavior, access unprotected functionality, or inject vulnerabilities such as XSS, SQL Injection, and Command Injection.
- **Business Logic Testing** – The application workflows were reviewed to identify opportunities to misuse or bypass intended processes. This included testing for logic errors, insufficient validation, and scenarios where typical constraints could be circumvented.

Scope

The assessment was conducted between May 16, 2026 and May 19, 2026. The re-assessment was conducted between May 16, 2026 and May 19, 2026. Testing was performed remotely.

The scope of the assessment was limited to the environments and targets below:

Application Details

Name	URL
vnsdvlds,mv	vesnvklmkdw

User Roles (Web application & API)

Role	Username
sndlvkn as	sedlkmvlk

Out Of Scope Endpoints

Name	URL
ensvkld	jvknewlsadvkm

Tools

Tool Name	Description
Burp Professional Pro	An advanced proxy for testing web security.
OpenSSL	An open-source package to assess security in transit.
Nmap	A tool used to discover hosts and services on a network.
SQLMap	Python-based CLI tool that identifies SQL injection issues.
Acunetix Pro	An automated scanner to perform authenticated scans on the web app / APIs.
cURL Utility	Command line utility to perform HTTP/s requests.

The following components and tests were out of scope for this review:

- Any applications and infrastructure external to the Nitin's Web Application & API.
- In cases where the Nitin's Web Application & API had inbound and/or outbound interfaces with:
- Another application, the interfaces, and communications were considered in scope.
- All other external elements were excluded.
- Supporting policies, procedures, and processes.
- Social Engineering.
- Software Development Life Cycle.

Assessment Limitation

The ever-changing technology landscape and the increasing sophistication of attacks against networked systems are reasons why no entity can truthfully claim to identify all security issues or guarantee the lifetime security of an organization’s network and applications. Note that this point-in-time assessment was based on a best-effort basis. It was also performed only in the environment provided by Nitin. Thus, changes to the environment may impact the applicability of the results provided herein.

SecurifyAI cannot guarantee 100% coverage for any security assessment.

Findings and Recommendation

Risk Classification

The remainder of this report describes the vulnerabilities that SecurifyAI identified as part of the assessment, their impact, and recommendations for resolving the vulnerabilities. To assist in determining the risk posed by these vulnerabilities, SecurifyAI leverages the OWASP Application Security Risk Rating Methodology. The observations have been categorized based on technical Impact and Likelihood, explained below. These impact and likelihood scores may be further modified by Nitin based on the business criticality of the target.

Measurement of Impact

Impact is an estimation of the potential damage via a successful exploit of a vulnerability. We'll use the following factors to help qualitatively determine the impact of a vulnerability.

- **Low Impact**: When most/all factors indicate limited consequences (e.g., non-sensitive data, no ability to alter/delete key data, and low victim count).
- **Medium Impact**: When about half of the factors suggest higher damage and half point to limited effects.
- **High Impact**: When most/all factors highlight significant damage (e.g., sensitive data loss, wide data corruption, and a large number of victims).

Factor	Description	Effect on Impact
Classification of Data Loss	If an attacker exploited this vulnerability, what type of data would they access or steal? Non-sensitive, non-customer-specific data would lower the impact.	Less critical data reduces the overall impact.
Data Integrity	Could the attacker modify or delete data they shouldn't? If yes, what kind of data could they corrupt?	Non-critical or limited modification rights reduce the impact of exploitation.
Blast Radius	How many users or systems would be affected by this vulnerability? For instance, an obscure service affects fewer users, while a public system affects many.	A small number of victims lowers the severity.

Measurement of Likelihood

Likelihood is a qualitative estimation of the probability of an attacker exploiting the vulnerability in question. In order to determine the likelihood of exploitation, we can consider the following

factors:

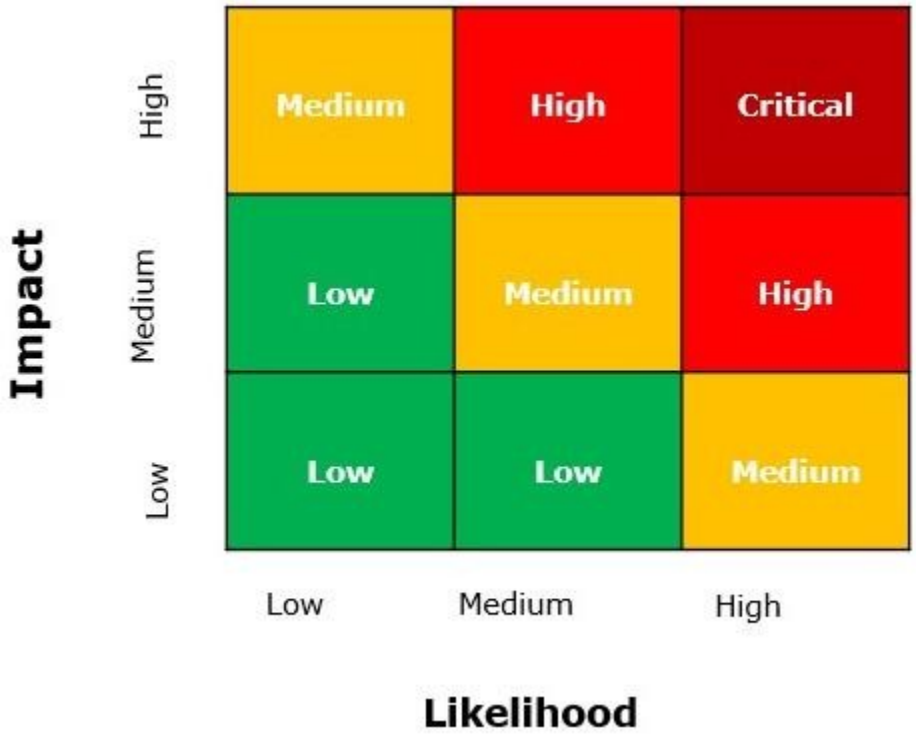
- **Low Likelihood**: Most/all factors suggest significant barriers to exploitation (e.g., complex skillset, limited attackers, high cost, or complex delivery).
- **Medium Likelihood**: About half the factors indicate ease of exploitation, while the other half show barriers.
- **High Likelihood**: Most/all factors point to easy and accessible exploitation (e.g., basic skills, low cost, and simple attack mechanisms).

Factor	Description	Effect on Likelihood
Required Skill Level of Attacker	How technically skilled does the attacker need to be to exploit the vulnerability? The higher the required skills, the lower the likelihood.	Complex skill requirements lower the chances of exploitation.
Number of Potential Attackers	How many attackers could potentially exploit this vulnerability? If access to the system is limited, fewer attackers can exploit it.	Fewer attackers reduce the probability of exploitation.
Cost of Exploit	How expensive are the tools or resources required for exploitation? High costs reduce accessibility for attackers.	Expensive tools lower the likelihood of exploitation.
Ease of Exploit	How simple is it for the attacker to exploit the vulnerability once discovered? Does it rely on external events or tools?	Complex or multi-step scenarios reduce the chances of successful exploitation.

Overall Risk

The following graph illustrates how Impact x Likelihood scores translate to overall Low, Medium, and High-risk ratings:

- **Low Impact + Low Likelihood** = Low Risk
- **High Impact + High Likelihood** = Critical Risk



Zero-risk Issues

There are cases where a particular issue doesn't have direct security impact or likelihood of exploitation. These could be issues related to documentation or semantics that are suggested based on the security assessors' technical and industry experience. Such issues shall be reported with an Informational risk rating.

Vulnerabilities

Summary

Below is a summary of the vulnerabilities discovered during the assessment, ranked in order of risk as determined.

Vulnerability	Risk
Auction Time Restriction Bypass via Client-Side Enforcement	**Critical**
Time-Based SQL Injection Leading to Denial of Service	**High**
Broken Access Control Leading to Account Takeover via API Endpoint	**Medium**

Detailed Vulnerabilities

Auction Time Restriction Bypass via Client-Side Enforcement

Risk: Critical

Description:

The auction system implements time restrictions solely at the client-side user interface level. When an auction's designated closing time is reached, the 'Bid' button in the UI becomes disabled, preventing users from submitting bids through standard interaction. However, the backend API endpoint responsible for processing bids does not perform server-side validation to confirm whether the auction is still active or has already concluded. This allows an attacker to bypass the client-side control by directly sending requests to the bidding API endpoint even after the auction has officially closed according to the client-side timer.

Affected URL:

- https://www.dev.blackrockgalleries.com/processingbidAjax.php

Impact and Likelihood:

- **Impact: High** – The impact of this vulnerability is significant, as it directly undermines the integrity and fairness of the auction process. Successful exploitation can lead to unauthorized bids being placed after an auction has closed, potentially altering auction outcomes, causing financial losses for legitimate bidders, and damaging the reputation and trustworthiness of the auction platform. This could result in disputes, chargebacks, and a loss of user confidence in the system's fairness and security.
- **Likelihood: High** – Exploitation of this vulnerability is highly feasible. An attacker requires minimal technical skill, primarily the ability to intercept or craft HTTP requests. By observing the legitimate bid request and replaying it after the client-side timer expires, or by directly crafting a request to the API endpoint, the time restriction can be easily circumvented. No complex authentication bypass or advanced techniques are required.

Steps to Reproduce:

Step 1: Access an active auction on the platform and observe the countdown timer.

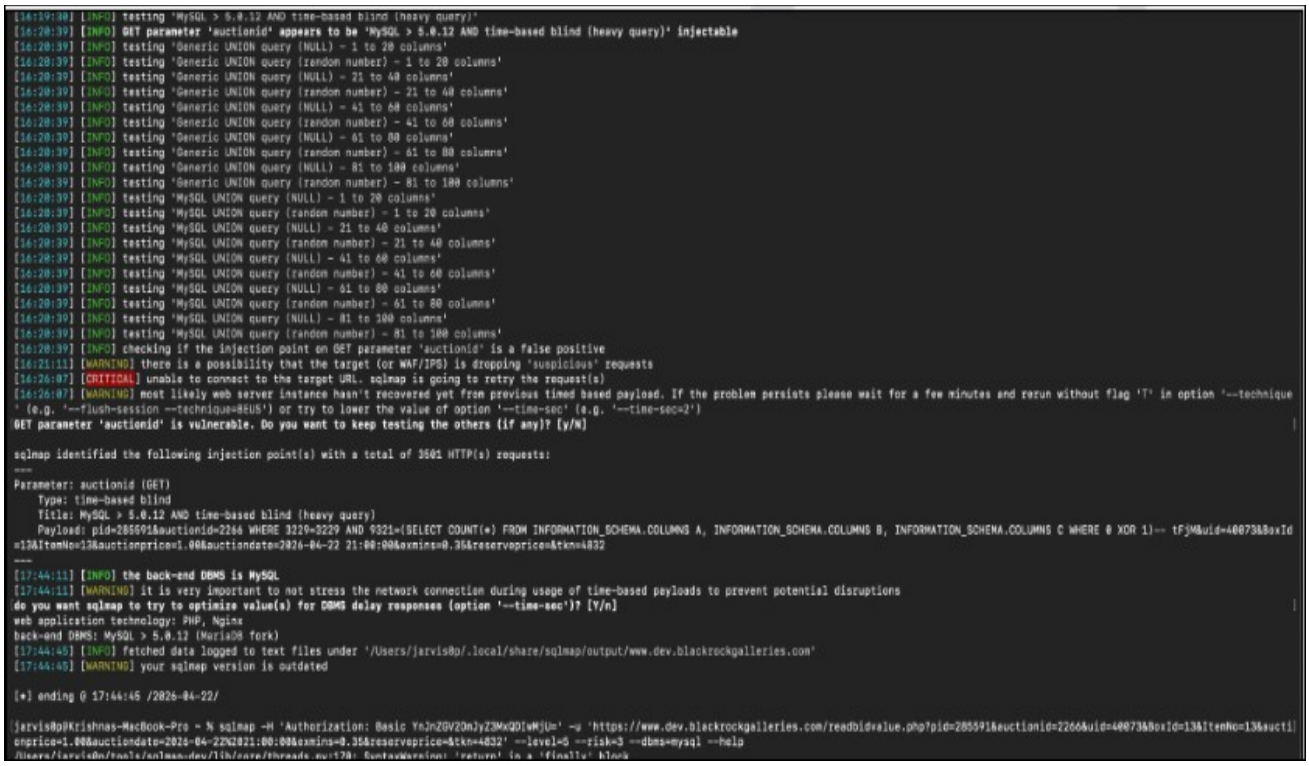


Fig 1: pic1

Step 2: Using a web proxy tool (e.g., Burp Suite), intercept a legitimate bid request made to the 'processingbidAjax.php' endpoint.

Step 3: Wait for the auction timer to expire, at which point the 'Bid' button in the user interface will become disabled.

Fig 3: pic3

Step 4: After the auction has closed client-side, resend the intercepted bid request (or a crafted equivalent) directly to the 'processingbidAjax.php' endpoint.

Step 5: Observe that the bid is successfully processed by the server, effectively bypassing the auction's closing time restriction.

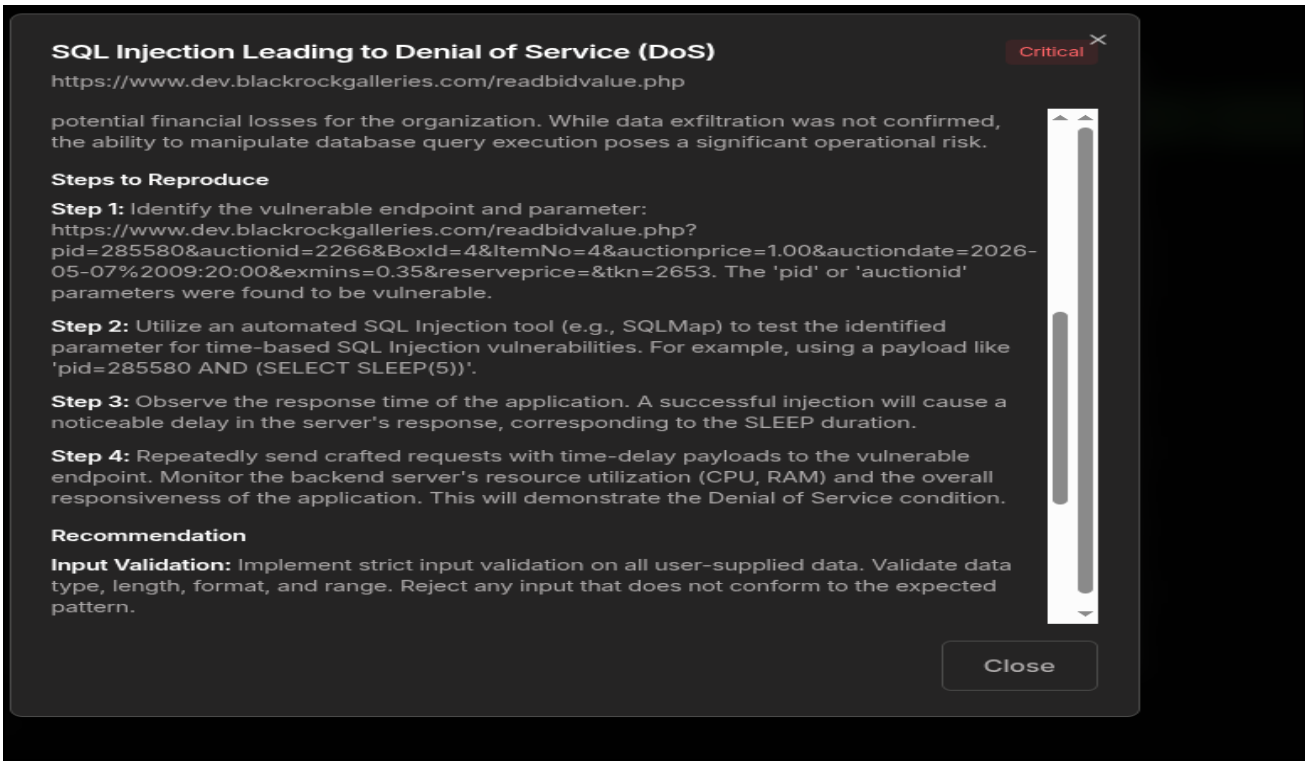


Fig 5: pic5

Recommendations:

- **Server-Side Validation:** Implement robust server-side validation for all auction-related actions, especially bid submissions. The server must independently verify the current status of an auction (e.g., open, closed, pending) before processing any bid requests.
- **Business Logic Enforcement:** Ensure that critical business logic, such as auction closing times, is enforced exclusively on the server-side and not solely reliant on client-side controls. Client-side controls should only be used for user experience enhancements, not for security enforcement.
- **API Security:** All API endpoints handling sensitive operations, like bidding, must include comprehensive checks against the current state of the system and user permissions. Do not trust data or state information provided by the client.
- **Input Validation:** Validate all parameters received by the 'processingbidAjax.php' endpoint against expected values and current auction rules, including the auction's active status.

References:

- https://cwe.mitre.org/data/definitions/602.html
- https://owasp.org/www-project-top-ten/2021/A04_2021-Insecure_Design

Time-Based SQL Injection Leading to Denial of Service

Risk: High

Description:

A Time-Based SQL Injection vulnerability was identified in the application's handling of user-supplied input within the bid-related API endpoints. The application fails to properly sanitize or validate input before incorporating it into backend database queries. This flaw allows an attacker to inject malicious SQL payloads that, while not necessarily enabling data exfiltration, can force the database server to execute resource-intensive operations. Automated testing with SQLMap confirmed the vulnerability, demonstrating that crafted requests could lead to high CPU and RAM utilization on the backend server, causing significant application degradation and potential Denial of Service (DoS) conditions.

Affected URL:

- https://www.dev.blackrockgalleries.com/readbidvalue.php?pid=285580&auctionid=2266&BoxId=4&ItemNo=4&auctionprice=1.00&auctiondate=2026-05-07%2009:20:00&exmins=0.35&reserveprice=&tkn=2653

Impact and Likelihood:

- **Impact: High** – The potential impact of this vulnerability is High. Successful exploitation can lead to a Denial of Service (DoS) condition, rendering the application unresponsive or unavailable to legitimate users. By forcing the database server to execute long-running queries, an attacker can consume significant CPU and RAM resources, causing severe performance degradation. While direct data exfiltration was not confirmed, the ability to manipulate database query execution poses a significant operational risk to the availability and reliability of the application.
- **Likelihood: High** – The likelihood of exploitation is High. The vulnerability was confirmed using automated tools (SQLMap) which successfully identified and exploited the time-based SQL Injection. The lack of proper input validation and sanitization makes it straightforward for an attacker to craft and inject malicious payloads, leading to predictable and reproducible resource exhaustion on the backend server. The affected parameter is directly exposed in the URL, simplifying the attack vector.

Steps to Reproduce:

Step 1: Identify the vulnerable endpoint and parameters. In this case, the 'pid' parameter in the 'readbidvalue.php' endpoint was found to be susceptible.

Fig 1: ic1

Step 2: Utilize an automated SQL Injection tool (e.g., SQLMap) or manually craft a time-based SQL Injection payload for the vulnerable parameter. For example, by appending a sleep-based payload to the 'pid' parameter.

Step 3: Send the crafted request to the application. Observe the response time and monitor the backend server's resource utilization (CPU, RAM). A successful injection will cause a noticeable delay in the response and an increase in server resource consumption.

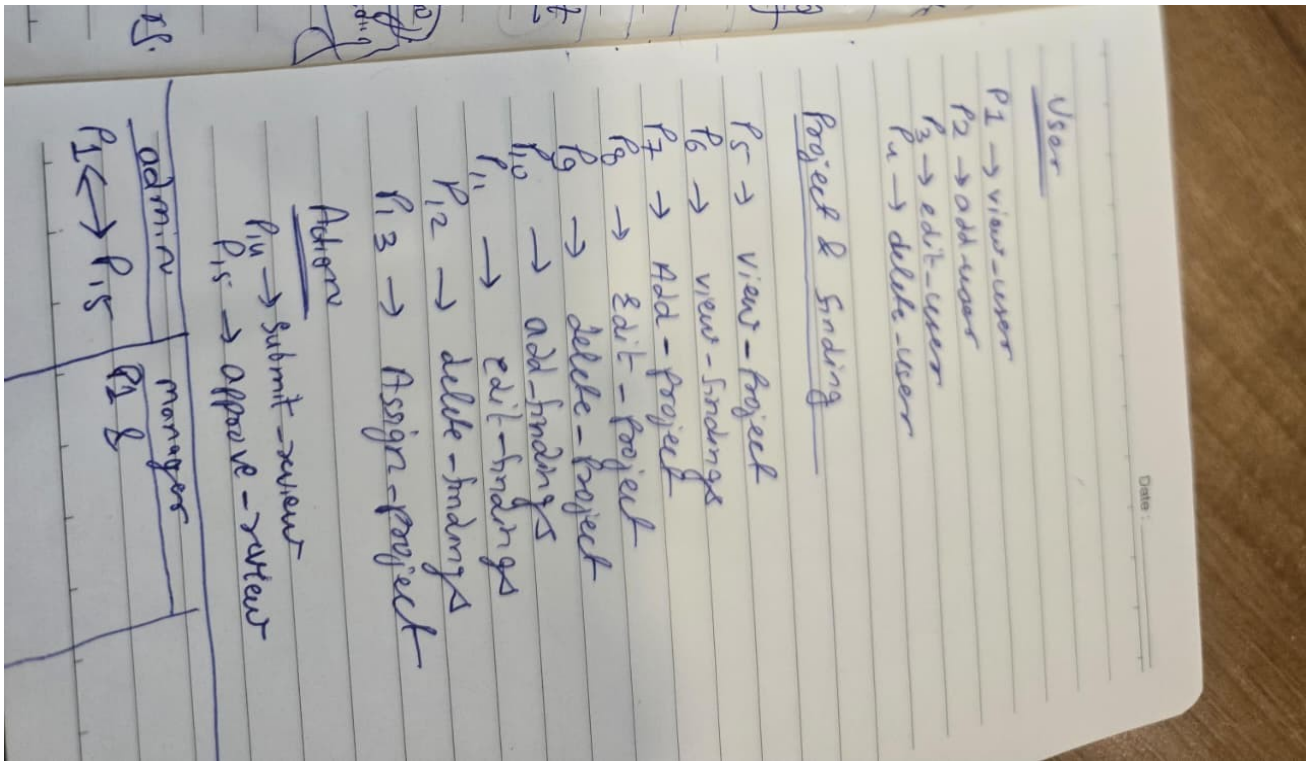


Fig 3: pic 3

Step 4: Repeatedly send such crafted requests to confirm the ability to induce resource exhaustion and application degradation, potentially leading to a Denial of Service.

Recommendations:

- **Input Validation and Sanitization:** Implement strict input validation and sanitization for all user-supplied data before it is processed or used in database queries. Ensure that only expected data types and formats are accepted.
- **Parameterized Queries/Prepared Statements:** Utilize parameterized queries or prepared statements for all database interactions. This ensures that user input is treated as data, not executable code, effectively preventing SQL Injection attacks.
- **Least Privilege:** Configure database users with the principle of least privilege, granting only the necessary permissions required for their specific functions. This limits the potential damage if an SQL Injection vulnerability is exploited.
- **Web Application Firewall (WAF):** Deploy and properly configure a Web Application Firewall (WAF) to detect and block common SQL Injection attack patterns. While not a primary defense, a WAF can provide an additional layer of security.
- **Error Handling:** Implement robust error handling that does not reveal sensitive database or application information to attackers. Generic error messages should be displayed to users.

References:

- https://owasp.org/www-community/attacks/SQL_Injection
- https://cwe.mitre.org/data/definitions/89.html

Broken Access Control Leading to Account Takeover via API Endpoint

Risk: Medium

Description:

The application implements role-based access control to restrict access to sensitive user information. While the normal application interface prevents lower-privileged users from viewing other users' details, the backend API endpoints responsible for retrieving user account information do not properly enforce authorization checks. A low-privileged authenticated user can directly access these API endpoints to retrieve sensitive information belonging to other users without authorization. The exposed response includes complete user account details, including usernames and passwords stored in clear text. This direct disclosure of credentials enables an attacker to authenticate as any user within the application, leading to full account compromise.

Affected URL:

- https://www.dev.blackrockgalleries.com/auction-admin/add_admin.php

Impact and Likelihood:

- **Impact: High** – The impact of this vulnerability is severe, as it allows for complete account takeover of any user within the application, including administrative accounts if present. An attacker can gain unauthorized access to sensitive data, perform actions on behalf of other users, and potentially escalate privileges. This can lead to data breaches, reputational damage, financial losses, and compromise the integrity and confidentiality of the application and its users.
- **Likelihood: High** – Exploitation of this vulnerability is highly likely as it only requires a low-privileged authenticated user to directly access a specific API endpoint. The lack of server-side authorization checks on the API, combined with the clear text disclosure of passwords, makes it straightforward for an attacker to enumerate and compromise user accounts with minimal effort.

Steps to Reproduce:

Step 1: Log in to the application as a low-privileged user (e.g., 'manager' role).

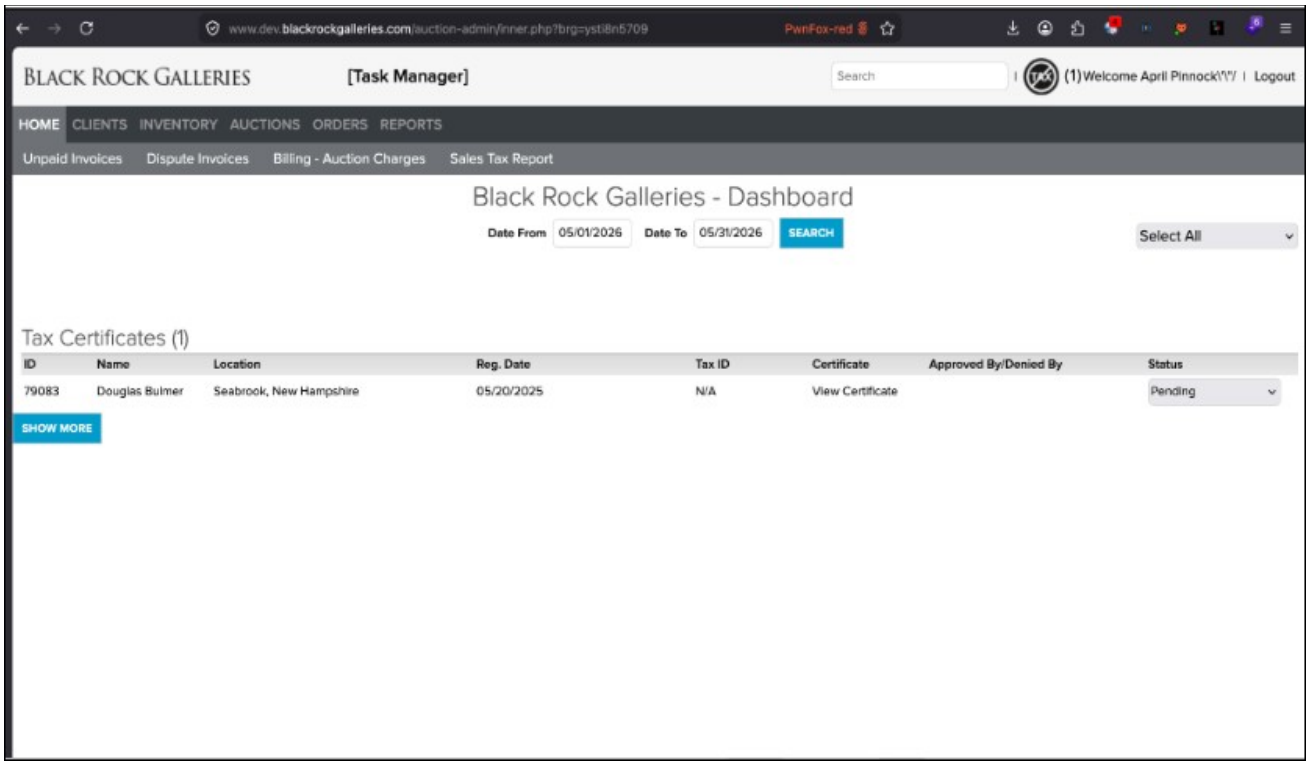


Fig 1: Lower user logged in into admin portal.

Step 2: Construct a direct request to the vulnerable API endpoint, for example: `https://www.dev.blackrockgalleries.com/auction-admin/add_admin.php?id=137` (where 'id' is a valid user ID).

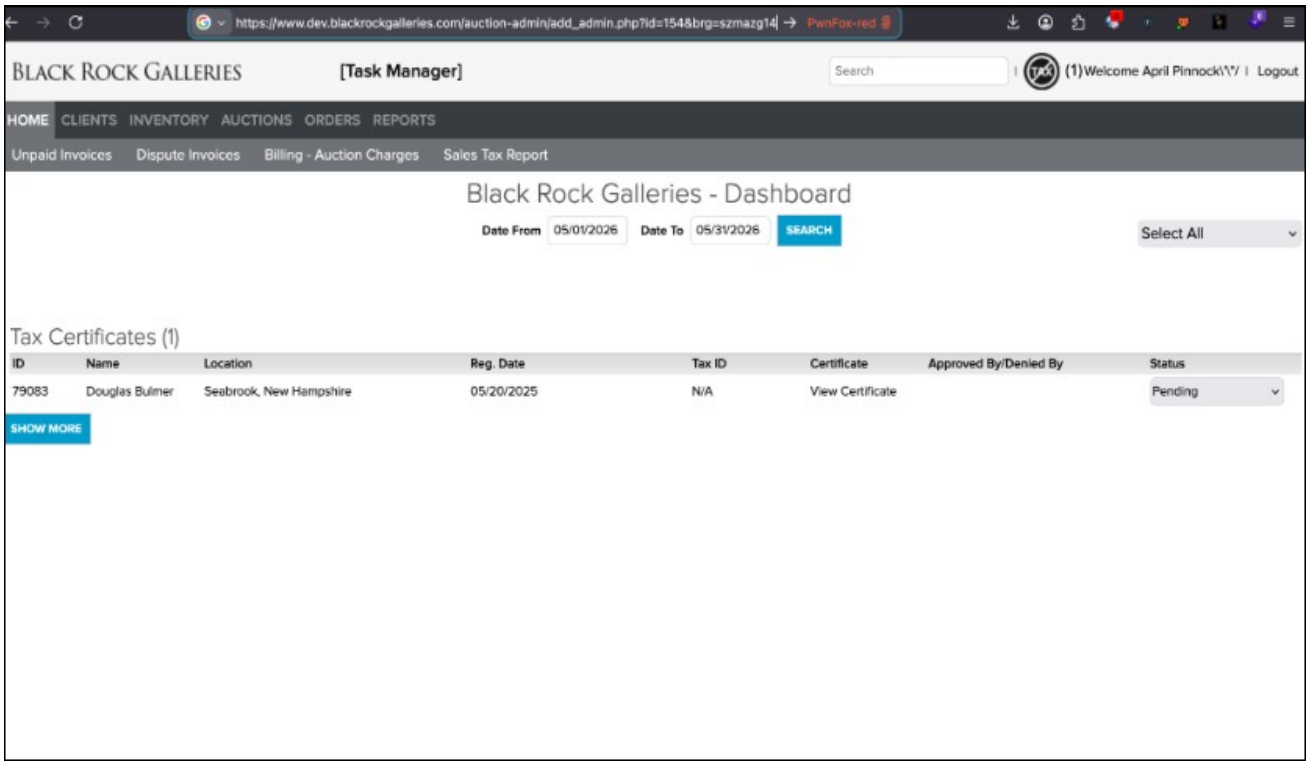


Fig 2: Forced browsing to view admin user details.

Step 3: Observe the response, which contains the complete user account details, including the username and password in clear text.

Step 4: Use the retrieved clear text credentials to log in as the compromised user, confirming account takeover.

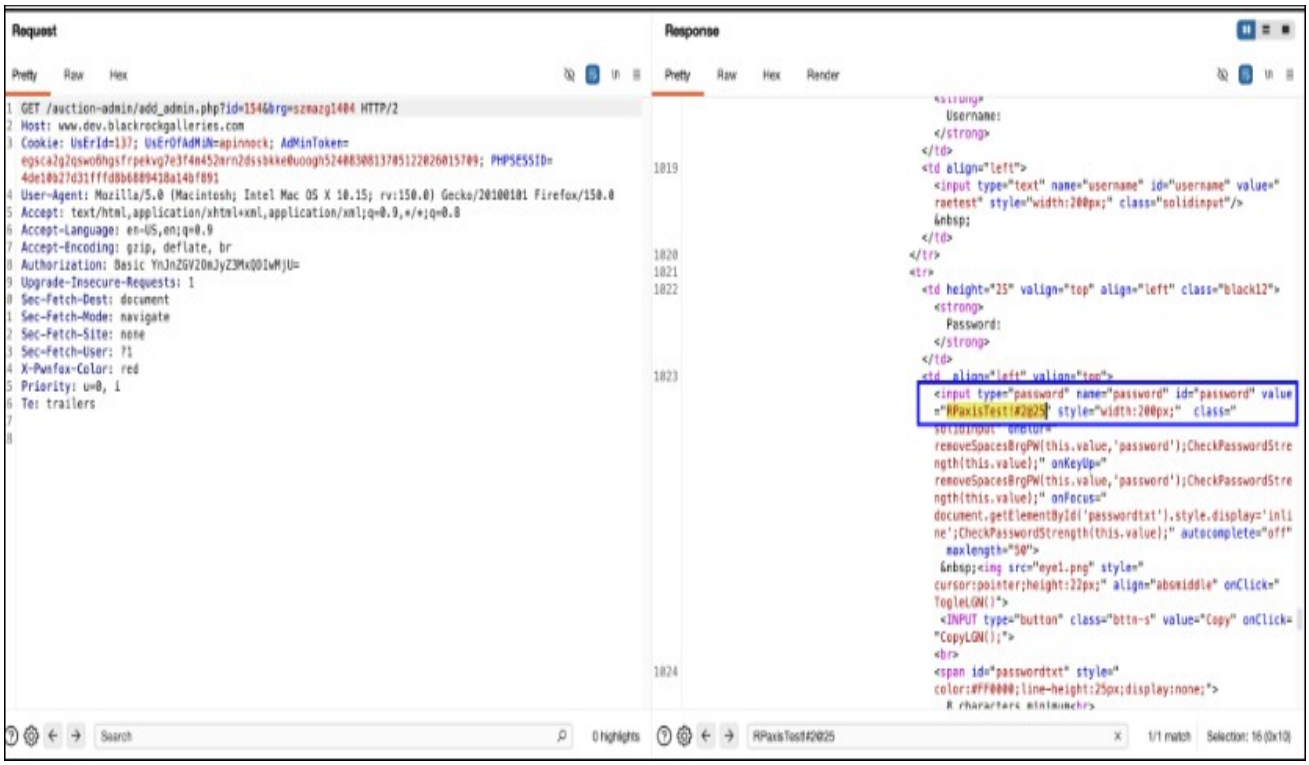


Fig 4: Shows clear text password revealed in HTTP response.

Recommendations:

- **Access Control:** Implement robust server-side authorization checks for all API endpoints. Ensure that every request to an API endpoint verifies the user's permissions and role before granting access to sensitive data or functionality. This should be enforced at the backend, independent of any client-side controls.
- **Secure Credential Storage:** Never store passwords in clear text. All user passwords must be stored using strong, one-way cryptographic hashing functions (e.g., bcrypt, Argon2, scrypt) with a unique salt for each password. This prevents the direct disclosure of credentials even if the database is compromised.
- **Principle of Least Privilege:** Ensure that users and roles are granted only the minimum necessary permissions to perform their intended functions. Restrict access to administrative functions and sensitive data to only authorized personnel.
- **Input Validation:** Implement strict input validation for all parameters, especially those used to identify resources (e.g., user IDs). This can help prevent Insecure Direct Object Reference (IDOR) vulnerabilities.

References:

- https://owasp.org/www-project-top-ten/2021/A01_2021-Broken_Access_Control
- https://cwe.mitre.org/data/definitions/284.html

- https://cwe.mitre.org/data/definitions/256.html

Appendix A

The appendix content below is static template material and will be rendered into reports that use this template.

Penetration Test Scope	
V1	Authentication Verification Requirements
V1.1	Password Security Requirements
1.1.1	Verify that user-set passwords are at least 12 characters in length
1.1.2	Verify that passwords 64 characters or longer are permitted
1.1.3	Verify that passwords can contain spaces and that truncation is not performed. Consecutive multiple spaces MAY optionally be coalesced
1.1.4	Verify that Unicode characters are permitted in passwords. A single Unicode code point is considered a character, so 12 emoji or 64 kanji characters should be valid and permitted
1.1.5	Verify users can change their password
1.1.7	Verify that passwords submitted during account registration, login, and password change are checked against a set of breached passwords either locally (such as the top 1,000 or 10,000 most common passwords that match the system's password policy) or using an external API. If using an API a zero-knowledge proof or other mechanism should be used to ensure that the plain text password is not sent or used in verifying the breach status of the password. If the password is breached, the application must require the user to set a new non non-breached password
1.1.8	Verify that a password strength meter is provided to help users set a stronger password
1.1.9	Verify that there are no password composition rules limiting the type of characters permitted. There should be no requirement for upper or lower case or numbers or special characters
1.1.10	Verify that "paste" functionality, browser password helpers, and external password managers are permitted
1.1.11	Verify that the user can choose to either temporarily view the entire masked password, or temporarily view the last typed character of the password on platforms that do not have this as native functionality
V1.2	General Authenticator Requirements

1.2.1	Verify that anti-automation controls are effective at mitigating breached credential testing, brute force, and account lockout attacks. Such controls include blocking the most common breached passwords, soft lockouts, rate limiting, CAPTCHA, ever-increasing delays between attempts, IP address restrictions, or risk-based restrictions such as location, first login on a device, recent attempts to unlock the account, or similar. Verify that no more than 100 failed attempts per hour is possible on a single account
1.2.2	Verify that the use of weak authenticators (such as SMS and email) is limited to secondary verification and transaction approval and not as a replacement for more secure authentication methods. Verify that stronger methods are offered before weak methods, that users are aware of the risks, or that proper measures are in place to limit the risks of account compromise
1.2.3	Verify that secure notifications are sent to users after updates to authentication details, such as credential resets, email or address changes, and logging in from unknown or risky locations. The use of push notifications - rather than SMS or email - is preferred, but in the absence of push notifications, SMS or email is acceptable as long as no sensitive information is disclosed in the notification
1.2.4	Verify impersonation resistance against phishing, such as the use of multi-factor authentication, cryptographic devices with intent (such as connected keys with a push to authenticate), or at higher ALL levels, client-side certificates
1.2.5	Verify that where a credential service provider (CSP) and the application verifying authentication are separated, mutually authenticated TLS is in place between the two endpoints
V1.3	Credential Recovery Requirements
1.3.1	Verify that a system-generated initial activation or recovery secret is not sent in clear text to the user
1.3.2	Verify password hints or knowledge-based authentication (so-called "secret questions") are not present
1.3.3	Verify password credential recovery does not reveal the current password in any way
1.3.4	Verify shared or default accounts are not present (e.g. "root", "admin", or "sa").
1.3.5	Verify that if an authentication factor is changed or replaced, the user is notified of this event
1.3.6	Verify forgotten passwords, and other recovery paths using a secure recovery mechanism, such as TOTP or another soft token, mobile push, or another offline recovery mechanism

1.3.7	Verify that if OTP or multi-factor authentication factors are lost, that evidence of identity proofing is performed at the same level as during enrolment
V1.4	Single or Multi-Factor One Time Verifier Requirements
1.4.1	Verify that time-based OTPs have a defined lifetime before expiring
1.4.2	Verify that symmetric keys used to verify submitted OTPs are highly protected, such as by using a hardware security module or secure operating system-based key storage
1.4.3	Verify that time-based OTP can be used only once within the validity period
1.4.4	Verify that if a time-based multi-factor OTP token is re-used during the validity period, it is logged and rejected with secure notifications being sent to the holder of the device
V2	Session Management Verification Requirements
V2.1	Fundamental Session Management Requirements
2.1.1	Verify the application never reveals session tokens in URL parameters or error messages
2.1.1	Verify the application generates a new session token on user authentication
2.1.2	Verify that session tokens possess at least 64 bits of entropy
2.1.3	Verify the application only stores session tokens in the browser using secure methods such as appropriately secured cookies or HTML 5 session storage
V2.2	Session Logout and Timeout Requirements
2.2.1	Verify that logout and expiration invalidate the session token, such that the back button or a downstream relying party does not resume an authenticated session, including across relying parties
2.2.2	If authenticators permit users to remain logged in, verify that re-authentication occurs periodically both when actively used or after an idle period. (C6) 30 days 12 hours or 30 minutes of inactivity, 2FA optional 12 hours or 15 minutes of inactivity, with 2FA
2.2.3	Verify that the application terminates all other active sessions after a successful password change and that this is effective across the application, federated login (if present), and any relying parties
2.2.4	Verify that users are able to view and log out of any or all currently active sessions and devices
V2.3	Cookie-based Session Management
2.3.1	Verify that cookie-based session tokens have the 'Secure' attribute set

2.3.2	Verify that cookie-based session tokens have the 'HttpOnly' attribute set
2.3.3	Verify that cookie-based session tokens utilize the 'SameSite' attribute to limit exposure to cross-site request forgery attacks
2.3.4	Verify that cookie-based session tokens use "__Host-" prefix (see references) to provide session cookie confidentiality
2.3.5	Verify that if the application is published under a domain name with other applications that set or use session cookies that might override or disclose the session cookies, set the path attribute in cookie-based session tokens using the most precise path possible
V2.4	Token-based Session Management
2.4.2	Verify the application uses session tokens rather than static API secrets and keys, except with legacy implementations
2.4.3	Verify that stateless session tokens use digital signatures, encryption, and other countermeasures to protect against tampering, enveloping, replay, null cipher, and key substitution attacks
V3	Access Control Verification Requirements
V3.1	Operation Level Access Control
3.1.2	Verify that the application or framework enforces a strong anti-CSRF mechanism to protect authenticated functionality, and effective anti-automation or anti-CSRF protects unauthenticated functionality
V4	Validation, Sanitization, and Encoding Verification Requirements Control Objective
V4.1	Input Validation Requirements
4.1.1	Verify that the application has defenses against HTTP parameter pollution attacks, particularly if the application framework makes no distinction about the source of request parameters (GET, POST, cookies, headers, or environment variables)
4.1.2	Verify that frameworks protect against mass parameter assignment attacks, or that the application has countermeasures to protect against unsafe parameter assignment, such as marking fields private or similar
4.1.3	Verify that all input (HTML form fields, REST requests, URL parameters, HTTP headers, cookies, batch files, RSS feeds, etc) is validated using positive validation (whitelisting)
4.1.4	Verify that structured data is strongly typed and validated against a defined schema including allowed characters, length and pattern (e.g. credit card numbers or telephone, or validating that two related fields are reasonable, such as checking that suburb and zip/postcode match)

4.1.5	Verify that URL redirects and forwards only allow whitelisted destinations, or show a warning when redirecting to potentially untrusted content
4.1.6	Verify that all untrusted HTML input from WYSIWYG editors or similar is properly sanitized with an HTML sanitizer library or framework feature
4.1.7	Verify that unstructured data is sanitized to enforce safety measures such as allowed characters and length
4.1.8	Verify that the application sanitizes user input before passing to mail systems to protect against SMTP or IMAP injection
4.1.9	Verify that the application avoids the use of eval() or other dynamic code execution features. Where there is no alternative, any user input being included must be sanitized or sandboxed before being executed
4.1.10	Verify that the application protects against template injection attacks by ensuring that any user input being included is sanitized or sandboxed
4.1.11	Verify that the application protects against SSRF attacks, by validating or sanitizing untrusted data or HTTP file metadata, such as filenames and URL input fields, using whitelisting of protocols, domains, paths, and ports
4.1.12	Verify that the application sanitizes, disables, or sandboxes user-supplied SVG scriptable content, especially as they relate to XSS resulting from inline scripts, and foreign Objects
4.1.13	Verify that the application sanitizes, disables, or sandboxes user-supplied scriptable or expression template language content, such as Markdown, CSS or XSL stylesheets, BBCode, or similar
V4.2	Output encoding and Injection Prevention Requirements
4.2.1	Verify that output encoding is relevant for the interpreter and context required. For example, use encoders specifically for HTML values, HTML attributes, JavaScript, URL Parameters, HTTP headers, SMTP, and others as the context requires, especially from untrusted inputs (e.g. names with Unicode or apostrophes, such as ねこ or O'Hara).
4.2.2	Verify that output encoding preserves the user's chosen character set and locale, such that any Unicode character point is valid and safely handled
4.2.3	Verify that context-aware, preferably automated - or at worst, manual - output escaping protects against reflected, stored, and DOM-based XSS
4.2.4	Verify that data selection or database queries (e.g. SQL, HQL, ORM, NoSQL) use parameterized queries, ORMs, entity frameworks, or are otherwise protected from database injection attacks

4.2.5	Verify that where parameterized or safer mechanisms are not present, context-specific output encoding is used to protect against injection attacks, such as the use of SQL escaping to protect against SQL injection
4.2.6	Verify that the application projects against JavaScript or JSON injection attacks, including for eval attacks, remote JavaScript includes, CSP bypasses, DOM XSS, and JavaScript expression evaluation.
4.2.7	Verify that the application protects against LDAP Injection vulnerabilities, or that specific security controls to prevent LDAP Injection have been implemented
4.2.8	Verify that the application protects against OS command injection and that operating system calls use parameterized OS queries or use contextual command line output encoding
4.2.9	Verify that the application protects against Local File Inclusion (LFI) or Remote File Inclusion (RFI) attacks.
4.2.10	Verify that the application protects against XPath injection or XML injection attacks
V4.3	Deserialization Prevention Requirements
4.3.1	Verify that serialized objects use integrity checks or are encrypted to prevent hostile object creation or data tampering
4.3.2	Verify that the application correctly restricts XML parsers to only use the most restrictive configuration possible and to ensure that unsafe features such as resolving external entities are disabled to prevent testE
4.3.3	Verify that deserialization of untrusted data is avoided or is protected in both custom code and third-party libraries (such as JSON, XML and YAML parsers)
4.3.4	Verify that when parsing JSON in browsers or JavaScript-based backends, JSON.parse is used to parse the JSON document. Do not use eval() to parse JSON
V5	Error Handling and Logging Verification Requirements
V5.1	Error Handling
5.1.1	Verify that a generic message is shown when an unexpected or security sensitive error occurs, potentially with a unique ID which support personnel can use to investigate
V6	Data Protection Verification Requirements
V6.1	Sensitive Private Data
6.1.1	Verify that sensitive data is sent to the server in the HTTP message body or headers, and that query string parameters from any HTTP verb do not contain sensitive data
V7	Communications Verification Requirements

V7.1	Communications Security Requirements
7.1.1	Verify that secured TLS is used for all client connectivity, and does not fall back to insecure or unencrypted protocols.
7.1.2	Verify using online or up-to-date TLS testing tools that only strong algorithms, ciphers, and protocols are enabled, with the strongest algorithms and ciphers set as preferred
7.1.3	Verify that old versions of SSL and TLS protocols, algorithms, ciphers, and configurations are disabled, such as SSLv2, SSLv3, or TLS 1.0 and TLS 1.1. The latest version of TLS should be the preferred cipher suite
V8	Business Logic Verification Requirements
8.1.1	Verify the application will only process business logic flows for the same user in sequential step order and without skipping steps
8.1.2	Verify the application will only process business logic flows with all steps being processed in realistic human time, i.e. transactions are not submitted too quickly
8.1.3	Verify the application has appropriate limits for specific business actions or transactions which are correctly enforced on a per-user basis
8.1.4	Verify the application has sufficient anti-automation controls to detect and protect against data exfiltration, excessive business logic requests, excessive file uploads, or denial of service attacks
8.1.5	Verify the application has business logic limits or validation to protect against likely business risks or threats, identified using threat modeling or similar methodologies
8.1.6	Tested for IDOR's vulnerabilities which would allow other users to retrieve any kind of sensitive information like api key, tokens etc.
8.1.7	Verified Privilege Escalation vulnerabilities which could allow low privilege users to perform arbitrary operations leading potential account takeovers.
8.1.8	Verified Privilege Escalation vulnerabilities which could allow users to perform CRUD Operations on behalf of victim users.
8.1.9	Checked for Information Disclosure vulnerabilities via IDORs which could result in users PII Disclosure (PII).
8.1.10	Checked for Broken Access Control issues via browsed forcing.
8.1.11	Checked for Mass Assignment vulnerability which could allow attackers to gain extra privileges.
V9	File and Resources Verification Requirements

V9.1	File Upload Requirements
9.1.1	Verify that the application will not accept large files that could fill up storage or cause a denial of service attack
V9.2	File Integrity Requirements
9.2.1	Verify that files obtained from untrusted sources are validated to be of expected type based on the file's content
V10	API and Web Service Verification Requirements
V10.1	Generic Web Service Security Verification Requirements
10.1.1	Verify that all application components use the same encodings and parsers to avoid parsing attacks that exploit different URI or file parsing behavior that could be used in SSRF and RFI attacks
10.1.2	Verify that access to administration and management functions is limited to authorized administrators
10.1.3	Verify API URLs do not expose sensitive information, such as the API key, session tokens, etc
10.1.4	Verify that authorization decisions are made at both the URI, enforced by programmatic or declarative security at the controller or router, and at the resource level, enforced by model-based permissions
10.1.5	Verify that requests containing unexpected or missing content types are rejected with appropriate headers (HTTP response status 406 Unacceptable or 415 Unsupported Media Type)
V10.2	RESTful Web Service Verification Requirements
V11	Configuration Verification Requirements
V11.1	Dependency
11.1.1	Verify that all components are up to date
11.1.2	Verify that all unneeded features, documentation, samples, configurations are removed, such as sample applications, platform documentation, and default or example users
11.1.3	Verify that if application assets, such as JavaScript libraries, CSS stylesheets or web fonts, are hosted externally on a content delivery network (CDN) or external provider, Subresource Integrity (SRI) is used to validate the integrity of the asset
V11.2	Unintended Security Disclosure Requirements

11.2.1	Verify that web or application server and framework error messages are configured to deliver user-actionable, customized responses to eliminate any unintended security disclosures
11.2.2	Verify that web or application server and application framework debug modes are disabled in production to eliminate debug features, developer consoles, and unintended security disclosures
11.2.3	Verify that the HTTP headers or any part of the HTTP response do not expose detailed version information of system components
V11.3	HTTP Security Headers Requirements
11.3.1	Verify that every HTTP response contains a content type header specifying a safe character set (e.g., UTF-8, ISO 8859-1).
11.3.2	Verify that all API responses contain Content-Disposition: attachment; filename="api.json" (or other appropriate filename for the content type).
11.3.3	Verify that a content security policy (CSPv2) is in place that helps mitigate impact of XSS attacks like HTML, DOM, JSON, and JavaScript injection vulnerabilities
11.3.4	Verify that all responses contain X-Content-Type-Options: nosniff
11.3.5	Verify that HTTP Strict Transport Security headers are included on all responses and for all subdomains, such as Strict-Transport-Security: max-age=15724800; include subdomains
11.3.6	Verify that a suitable "Referrer-Policy" header is included, such as "no-referrer" or "same-origin
11.3.7	Verify that a suitable X-Frame-Options or Content-Security-Policy: frame ancestors header is in use for sites where content should not be embedded in a third-party site
V11.4	Validate HTTP Request Header Requirements
11.4.1	Verify that the application server only accepts the HTTP methods in use by the application or API, including pre-flight OPTIONS
11.4.2	Verify that the supplied Origin header is not used for authentication or access control decisions, as the Origin header can easily be changed by an attacker
11.4.3	Verify that the cross-domain resource sharing (CORS) Access-Control-AllowOrigin header uses a strict white list of trusted domains to match against and does not support the "null" origin
11.4.4	Verify that HTTP headers added by a trusted proxy or SSO devices, such as a bearer token, are authenticated by the application